

DOCUMENTAÇÃO TÉCNICA

Axel CRM

Plataforma multi-tenant de gestão comercial, operacional e financeira — Angular 18, Spring Boot 4 e PostgreSQL.

Escopo: arquitetura, segurança e multi-tenancy, modelo de dados, API REST, frontend, regras de negócio, ambiente e deploy, guia de desenvolvimento e pontos de atenção.

Base: comportamento verificado no código-fonte do repositório `axel-crm`, branch `master`.

Gerado em: 07 de agosto de 2026

Sumário

01 — Visão geral

- O que é o Axel CRM
- Usuários
- Papéis
- Stack
- Números do sistema
- Idioma e formatação

02 — Arquitetura

- Visão macro
- Backend — organização de pacotes
- Fluxo de uma requisição autenticada
- Padrões de código no backend
- Tratamento de erros
- Frontend — organização
- Camada de dados do frontend

03 — Segurança e multi-tenancy

- Autenticação
- Rotas públicas
- Isolamento multi-tenant
- Autorização por papel
- CORS
- Armazenamento de sessão no cliente
- LGPD
- Trilha de auditoria

04 — Modelo de dados

- Convenções comuns
- Tabelas por domínio

Entidades principais

Enums

Views analíticas

Migrations

05 — API REST

Convenções

Autenticação e usuários

CRM

Operações

Financeiro

Dashboard e analytics

Transversais

Endpoints públicos

06 — Frontend

Estrutura de rotas

Shell

Componentes compartilhados

Camada core

Telas com layout próprio

Configuração de ambiente

Design system

07 — Regras de negócio

Funil comercial

Lead scoring

Motor de pipeline

Propostas

Comissões

Apontamento de horas

Campanhas

Plano de contas

08 — Ambiente e deploy

Pré-requisitos

Execução local

Documentação da API em execução

Perfis de configuração

Variáveis de ambiente

Migrations

Deploy no Render

09 — Guia de desenvolvimento

Como adicionar um módulo CRUD completo

Convenções obrigatórias

Testes

Build

Commits

Onde procurar cada coisa

10 — Pontos de atenção

Crítico

Alto

Médio

Baixo

01 — Visão geral

O que é o Axel CRM

O Axel CRM é uma plataforma multi-tenant que reúne, num único workspace autenticado, três domínios que normalmente ficam em sistemas separados:

- **Comercial** — prospecção, parceiros, clientes, contatos, leads, negócios e pipelines
- **Operacional** — projetos, produtos, contratos, processos jurídicos, tarefas, propostas, campanhas, tickets, documentos e agenda
- **Financeiro** — faturamento, transações, plano de contas, contas bancárias, apontamento de horas, comissões e relatórios

O objetivo é permitir que um operador percorra o ciclo completo — prospect → lead → negócio → proposta → contrato → projeto → faturamento → relatório — sem trocar de sistema.

Usuários

Operadores internos (uso primário). Times mistos de vendas, operações e financeiro compartilhando a mesma organização. Uso desktop-first, diário e prolongado.

Superfícies secundárias:

Superfície	Rota	Autenticação
Portal do cliente	<code>/portal/client</code>	Sessão do portal
Portal do parceiro	<code>/portal/partner</code>	Sessão do portal
Proposta pública	<code>/public/proposals/:token</code>	Token UUID na URL, sem login

Papéis

O enum `Role` define sete papéis: `SUPER_ADMIN`, `ADMIN`, `MANAGER`, `SALES`, `SUPPORT`, `USER` e `VIEWER`.

Hoje o controle efetivo de acesso por papel acontece no frontend, através do `adminGuard`, que restringe as áreas **Usuários** (`/users`) e **Integrações** (`/integrations`) a `ADMIN` e `SUPER_ADMIN`. A API autentica todas as rotas privadas, mas não aplica autorização por papel — veja [Pontos de atenção](#).

Stack

Camada	Tecnologia
Frontend	Angular 18 (standalone components), Angular Material 18, Chart.js 4, RxJS 7

Camada	Tecnologia
Backend	Spring Boot 4.0.7, Spring Security, Spring Data JPA, Lombok
Banco	PostgreSQL, migrations via Flyway
Autenticação	JWT assinado com HMAC256 (biblioteca Auth0 <code>java-jwt</code>)
PDF	OpenPDF (propostas e faturas)
Documentação de API	SpringDoc OpenAPI / Swagger UI
Runtime	Java 21, Node.js 20+
Deploy	Render (Blueprint <code>render.yaml</code>), backend em Docker

Números do sistema

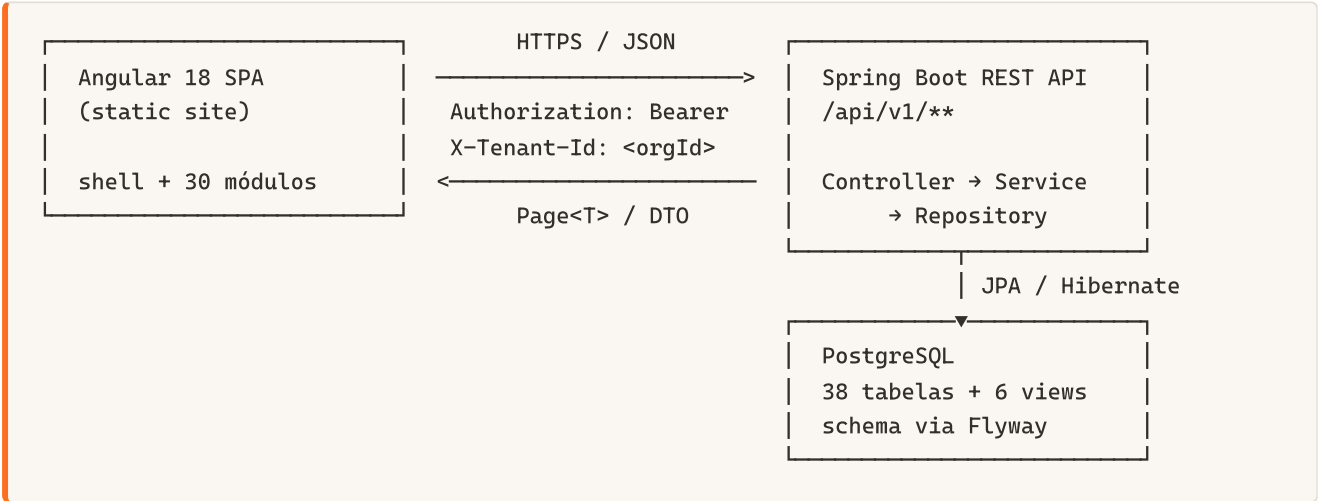
Item	Quantidade
Controllers REST	43
Services de domínio	42
Entidades JPA	38
Migrations Flyway	28 versionadas + 1 repetível (seed)
Tabelas	38
Views analíticas	6
Módulos de frontend (rotas)	30

Idioma e formatação

A interface é integralmente pt-BR: moeda em BRL, datas em `dd/mm/aaaa` e terminologia de negócio brasileira. Mensagens de erro da API misturam português e inglês conforme o ponto do código — as validações de regra de negócio (`BadRequestException`) estão em português.

02 — Arquitetura

Visão macro



Backend — organização de pacotes

Raiz: `backend/src/main/java/com/axelcrm/`

Pacote	Responsabilidade
<code>auth/</code>	Módulo vertical de autenticação: controller, dto, entity, repository, security, service
<code>commons/</code>	<code>BaseEntity</code> , <code>Organization</code> , enum <code>Role</code> , exceções e <code>GlobalExceptionHandler</code>
<code>config/</code>	CORS, OpenAPI, auditoria JPA, execução programática do Flyway
<code>controller/</code>	Controllers REST dos módulos de negócio
<code>dto/</code>	Records de request e response
<code>entity/</code>	Entidades JPA e <code>entity/enums/</code>
<code>repository/</code>	Interfaces Spring Data JPA
<code>service/</code>	Regras de negócio
<code>analytics/</code>	Repositório que lê as views analíticas

O módulo `auth` segue organização vertical (fatia completa por feature); os demais módulos seguem organização horizontal (por tipo de camada). Ao criar código novo, mantenha o padrão do módulo em que você está mexendo.

Fluxo de uma requisição autenticada

1. `JwtAuthenticationFilter` lê o header `Authorization: Bearer <token>`, valida a assinatura HMAC256 e extrai `userId`, `role` e `organizationId`. Popula o `SecurityContextHolder` com um `UsernamePasswordAuthenticationToken` cujo *principal* é o `UUID` do usuário, e grava o `organizationId` como atributo da requisição.
2. `TenantFilter` lê esse atributo e o coloca no `TenantContext` (`ThreadLocal<UUID>`), limpando-o no `finally`.
3. **Controller** obtém o tenant com `TenantContext.getOrganizationId()` e o repassa explicitamente ao service.
4. **Service** executa a regra e chama o repositório sempre filtrando por `organization_id`.
5. **Repository** aplica o filtro por convenção de nomes (`...AndOrganization_IdAndDeletedAtIsNull`).
6. A resposta é serializada como DTO (`record`), nunca como entidade JPA.

Ambos os filtros limpam seu estado no `finally`, o que é obrigatório em pools de threads reutilizadas.

Padrões de código no backend

Controllers são finos: extraem o tenant, delegam ao service e devolvem `ResponseEntity`. Anotados com `@Tag` e `@Operation` para alimentar o Swagger.

Services concentram a regra de negócio, são transacionais (`@Transactional`) e fazem o mapeamento entidade → DTO.

DTOs são Java `records` no padrão `XxxRequest` / `XxxResponse`, validados com `@Valid` e Jakarta Validation.

Entidades estendem `BaseEntity`, que fornece:

- `id` (`UUID`, gerado pela aplicação)
- `organization` (`@ManyToOne` obrigatório)
- `createdAt` / `updatedAt` (`@CreationTimestamp` / `@UpdateTimestamp`)
- `deletedAt` — soft delete; nenhuma exclusão é física
- um hook `@PrePersist` que preenche a organização a partir do `TenantContext` quando ela não foi setada explicitamente

Exclusão é sempre lógica: os métodos `delete` preenchem `deletedAt` e todas as consultas filtram por `DeletedAtIsNull`.

Tratamento de erros

`GlobalExceptionHandler` (`@RestControllerAdvice`) converte exceções em JSON com formato uniforme:

```
{ "error": "mensagem", "status": 400, "timestamp": "2026-08-07T10:00:00" }
```


Exceção	HTTP
<code>ResourceNotFoundException</code>	404
<code>BadRequestException</code>	400
<code>MethodArgumentNotValidException</code>	400 (inclui <code>details</code> com os campos inválidos)
<code>DataIntegrityViolationException</code>	400
<code>HttpMessageNotReadableException</code>	400
<code>BadCredentialsException</code>	401

Frontend — organização

Raiz: `frontend/src/app/`

Pasta	Responsabilidade
<code>core/</code>	Guards, interceptors, modelos e serviços transversais
<code>shared/</code>	<code>ListPageComponent</code> , <code>FormDialogComponent</code> , <code>TimelineComponent</code>
<code>shell/</code>	Layout autenticado: sidebar, toolbar, notificações, cronômetro global
<code>features/</code>	Um diretório por módulo, com <code>*.routes.ts</code> e componentes lazy-loaded

Todos os componentes são **standalone**; todas as rotas de módulo usam `loadChildren` / `loadComponent`, então cada módulo vira um chunk separado no build.

Camada de dados do frontend

`BaseService<T>` centraliza as chamadas HTTP genéricas — `getPage`, `getById`, `create`, `update`, `delete` — sobre `environment.apiUrl`. Serviços de módulo herdam dele e passam o path do recurso. Fluxos específicos (timeline, detalhe de cliente, detalhe de lead) têm serviços dedicados em `core/services/`.

Dois interceptors funcionais são registrados em `app.config.ts`:

- `authInterceptor` — injeta `Authorization: Bearer <token>` e, em resposta 401, faz logout e redireciona para `/login`
- `tenantInterceptor` — injeta o header `X-Tenant-Id` com o `organizationId` armazenado

O `X-Tenant-Id` é informativo: o backend **não** o utiliza. O tenant efetivo vem sempre do claim `organizationId` dentro do JWT.

03 — Segurança e multi-tenancy

Autenticação

A API é **stateless** (`SessionCreationPolicy.STATELESS`), sem sessão HTTP e com CSRF desabilitado — coerente com autenticação por token em SPA.

Token JWT

Gerado por `JwtUtil` com algoritmo **HMAC256**. Claims:

Claim	Conteúdo
<code>userId</code>	UUID do usuário
<code>email</code>	E-mail do usuário
<code>role</code>	Nome do papel (<code>ADMIN</code> , <code>SALES</code> , ...)
<code>organizationId</code>	UUID da organização (tenant)
<code>iat</code> / <code>exp</code>	Emissão e expiração

Validade padrão: **86.400.000 ms (24 h)**, configurável por `JWT_EXPIRATION`.

Registro e login

`POST /api/v1/auth/register` cria **uma nova organização** e o primeiro usuário dela com papel `ADMIN`. Não existe fluxo de convite de usuário para organização existente via `/auth/register`; usuários adicionais são criados pelo módulo administrativo `/api/v1/users`.

`POST /api/v1/auth/login` valida e-mail e senha (BCrypt), recusa usuário inativo e devolve o token junto dos dados de sessão.

Ambos respondem com `LoginResponse`: `token`, `userId`, `userName`, `email`, `role`, `organizationId`, `organizationName`.

Senhas são gravadas com `BCryptPasswordEncoder`.

Rotas públicas

Declaradas em `SecurityConfig.PUBLIC_PATHS`:

```
/                                /api/v1/auth/**
/api/v1/proposals/public/**      /swagger-ui/**      /swagger-ui.html
/v3/api-docs                    /v3/api-docs/**
/actuator/health                /actuator/info      /error
```

`/error` é público de propósito: sem essa liberação, erros do servidor eram mascarados como falha de CORS na SPA.

Todas as demais rotas exigem token válido (`anyRequest().authenticated()`).

Isolamento multi-tenant

O isolamento é **em nível de aplicação**, não de banco (não há Row Level Security nem schema por tenant). A garantia depende de três mecanismos combinados:

1. **TenantContext** — `ThreadLocal<UUID>` populado pelo `TenantFilter` a partir do claim do JWT e limpo ao fim da requisição.
2. **Passagem explícita** — o `organizationId` é passado do controller ao service e do service ao repositório em toda operação de leitura e escrita.
3. **Consultas filtradas** — os métodos de repositório carregam o filtro no nome, por exemplo:

```
findByIdAndOrganization_IdAndDeletedAtIsNull(UUID id, UUID organizationId)
```

Além disso, `BaseEntity.prePersist()` atribui a organização a partir do `TenantContext` quando a entidade é persistida sem organização explícita, evitando registros órfãos.

Consequência prática: um repositório novo que exponha um `findById` sem o filtro de organização abre um vazamento entre tenants. Toda consulta nova precisa do filtro por `organization_id`.

Autorização por papel

O enum `Role` tem sete valores: `SUPER_ADMIN`, `ADMIN`, `MANAGER`, `SALES`, `SUPPORT`, `USER`, `VIEWER`. O papel entra no token como authority do Spring Security.

No **frontend**, `adminGuard` bloqueia `/users` e `/integrations` para quem não é `ADMIN` ou `SUPER_ADMIN`, redirecionando ao dashboard.

No **backend**, não há `@PreAuthorize` nem `hasRole` em nenhum controller: qualquer usuário autenticado da organização pode chamar qualquer endpoint privado dela. Guard de frontend não é controle de acesso — está registrado em **Pontos de atenção**.

CORS

`CorsConfig` permite qualquer origem (`allowedOriginPatterns: *`), os métodos `GET`, `POST`, `PUT`, `DELETE`, `PATCH`, `OPTIONS`, qualquer header, com `allowCredentials: true` e cache de preflight de 1 h. Expõe `Authorization` e `X-Tenant-Id`.

Armazenamento de sessão no cliente

`AuthService` grava em `localStorage`:

Chave	Conteúdo
<code>crm_token</code>	JWT
<code>crm_user</code>	JSON com id, nome, e-mail, papel e organização
<code>crm_tenant</code>	<code>organizationId</code>

O logout remove as três chaves e redireciona para `/login`. Não há refresh token: expirado o JWT, a próxima chamada retorna 401 e o `authInterceptor` força novo login.

LGPD

`LgpdService` e `LgpdController` (`/api/v1/lgpd`) cobrem três obrigações:

Endpoint	Função
<code>POST /consent</code>	Registra ou atualiza consentimento por e-mail e tipo (tabela <code>lgpd_consent</code>)
<code>GET /export</code>	Portabilidade: exporta os dados pessoais de um titular (leads, clientes, contatos, consentimentos)
<code>DELETE /forget</code>	Direito ao esquecimento: exclusão / anonimização dos dados do titular

O consentimento é único por combinação de e-mail + tipo dentro da organização: um novo registro do mesmo par atualiza o existente.

Trilha de auditoria

`AuditLogService` grava em `audit_logs` a organização, o usuário, a ação, a entidade afetada, o valor anterior e o novo. É acionado em operações sensíveis do domínio comercial — transição de estágio (`STAGE_TRANSITION`), perda (`DEAL_LOST`), reabertura (`DEAL_REOPENED`) e conversão de lead (`CONVERT`). A leitura é exposta em `/api/v1/audit-logs`.

04 — Modelo de dados

O schema é gerenciado exclusivamente pelo **Flyway** (`backend/src/main/resources/db/migration/`). O Hibernate roda com `ddl-auto: none` — ele nunca altera o banco.

Convenções comuns

Toda tabela de negócio herda a estrutura de `BaseEntity`:

Coluna	Tipo	Observação
<code>id</code>	<code>UUID</code>	Chave primária, gerada pela aplicação
<code>organization_id</code>	<code>UUID</code>	FK obrigatória para <code>organizations</code> — chave do isolamento multi-tenant
<code>created_at</code>	<code>TIMESTAMP</code>	Preenchido no insert
<code>updated_at</code>	<code>TIMESTAMP</code>	Atualizado a cada update
<code>deleted_at</code>	<code>TIMESTAMP</code>	Soft delete: <code>NULL</code> significa registro ativo

Exceções: `organizations` e `users` não têm `organization_id` no mesmo sentido (`users` referencia a organização, `organizations` é a raiz do tenant).

Tabelas por domínio

Núcleo

Tabela	Descrição
<code>organizations</code>	Tenant. Nome, domínio único, documento, contato, endereço, flag <code>active</code>
<code>users</code>	Usuário. E-mail único global, senha BCrypt, papel, avatar, flag <code>active</code>
<code>audit_logs</code>	Trilha de auditoria de ações sensíveis
<code>notifications</code>	Notificações do usuário (lidas / não lidas)
<code>integrations</code>	Configuração de integrações externas
<code>google_tokens</code>	Tokens OAuth2 do Google por usuário
<code>lgpd_consent</code>	Registro de consentimentos LGPD

Comercial (CRM)

Tabela	Descrição
prospects	Topo do funil, antes de virar lead
partners	Parceiros e indicadores de negócio
leads	Lead com origem, estágio, score e valor estimado
lead_notes	Anotações do lead
clients	Cliente convertido, com status, segmento e responsável
client_notes	Anotações do cliente
client_attachments	Anexos do cliente
contacts	Pessoas de contato vinculadas a clientes
pipelines	Funil de vendas
pipeline_stages	Estágios ordenados por position dentro do pipeline
deals	Negócio: valor, pipeline, estágio, cliente e responsável
deal_stage_history	Histórico de permanência em cada estágio, com duração e motivo
campaigns	Campanhas de marketing
campaign_recipients	Destinatários de cada campanha
messages	Comunicações registradas (e-mail, WhatsApp, SMS, ligação, Instagram)

Operações

Tabela	Descrição
proposals	Proposta comercial, com código, token público e rateio de comissões
proposal_items	Itens da proposta
contracts	Contratos
projects	Projeto de entrega, com orçamento, custo e dados periciais
products	Catálogo de produtos e serviços
tasks	Tarefas
calendar_events	Eventos de agenda
legal_processes	Processos jurídicos (número CNJ)
documents	Documentos

Tabela	Descrição
<code>support_tickets</code>	Tickets de suporte
<code>time_entries</code>	Apontamento de horas

Financeiro

Tabela	Descrição
<code>invoices</code>	Fatura: número único, emissão, vencimento, pagamento, subtotal, impostos, desconto, total
<code>financial_transactions</code>	Lançamentos de entrada, saída, transferência, estorno e ajuste
<code>chart_of_accounts</code>	Plano de contas hierárquico
<code>bank_accounts</code>	Contas bancárias
<code>commissions</code>	Comissões apuradas
<code>commission_rules</code>	Regras de cálculo de comissão

Entidades principais

Client

`name`, `email`, `phone`, `document`, `companyName`, `website`, `address`, `city`, `state`, `zipCode`, `country`, `industry`, `notes`, `active`, `status` (`ClientStatus`), `serviceType`, `assignedTo` (usuário), `contacts` (lista).

Lead

`name`, `email`, `phone`, `company`, `position`, `source` (`LeadSource`), `stage` (`LeadStage`), `notes`, `score`, `estimatedValue`, `lastContactAt`, `convertedAt`, `assignedTo`, `convertedClient` (cliente resultante da conversão), `partner`.

Deal

`title`, `description`, `value`, `pipeline`, `stage`, `client`, `contact`, `assignedTo`, `expectedCloseDate`, `closedAt`, `won`.

O campo `won` é um `Boolean` com **três estados semânticos**:

Valor	Significado
<code>null</code>	Negócio aberto, em andamento
<code>true</code>	Ganho

Valor	Significado
false	Perdido

Proposal

proposalCode, publicToken (UUID usado no link público), title, description, status (ProposalStatus), issueDate, validUntil, totalAmount, discountAmount, approvedAt, client, assignedTo, items, deal, partner e o bloco de rateio de comissão: captureUser / captureRate, sellerUser / sellerRate, collaboratorUser / collaboratorRate, partnerRate.

Há ainda o bloco jurídico e pericial (migration V28):

Campo	Tipo	Descrição
lawyerContact	FK → contacts	Advogado vinculado, escolhido entre os contatos cadastrados
lawyerName	VARCHAR(200)	Nome do advogado quando ele ainda não existe no cadastro
referralSource	LeadSource	Origem da indicação — quem indicou continua sendo partner
expertUser	FK → users	Perito responsável; não participa do rateio de comissão
technicalManagerUser	FK → users	Responsável técnico; não participa do rateio de comissão
project	FK → projects	Projeto vinculado à proposta

lawyerContact e lawyerName são mutuamente exclusivos: com o contato vinculado, o nome livre é descartado e a leitura devolve o nome do contato (resolveLawyerName()).

O vínculo com projeto é bidirecional por caminhos distintos: Proposal.project aponta para o projeto atual, e Project.sourceProposalId registra a proposta que originou o projeto criado automaticamente.

Project

name, description, startDate, endDate, budget, cost, status (string livre, padrão "PLANEJAMENTO"), client, manager, sourceProposalId (rastreia a proposta que originou o projeto), legalProcess, cnjNumber, expertType, paymentStatus, deliveryDeadline.

Os campos cnjNumber, expertType e legalProcess refletem o uso do sistema em perícias e trabalhos jurídicos.

Invoice

invoiceNumber (único), client, contract, issueDate, dueDate, paidDate, status (string, padrão "DRAFT"), subtotal, taxAmount, discountAmount, total, paidAmount, paymentMethod, notes.

Enums

Enum	Valores
LeadSource	WEBSITE, SOCIAL_MEDIA, REFERRAL, EMAIL, PHONE, EVENT, ADVERTISEMENT, PARTNER, OTHER
LeadStage	NEW, CONTACTED, QUALIFIED, PROPOSAL, NEGOTIATION, CONVERTED, LOST, ARCHIVED
ProspectStage	PROSPECTING, CONTACTED, WAITING_REPLY
ClientStatus	NEW, CONTACTED, QUALIFIED, WAITING_RESPONSE, CLOSED_WON, CLOSED_LOST, DISCARDED, ONBOARDING, ACTIVE, INACTIVE
ProposalStatus	DRAFT, SENT, VIEWED, NEGOTIATING, ACCEPTED, REJECTED, EXPIRED
TaskStatus	PENDING, IN_PROGRESS, BLOCKED, COMPLETED, CANCELLED
TransactionType	INCOME, EXPENSE, TRANSFER, REFUND, ADJUSTMENT
ChartOfAccountType	RECEITA, DESPESA, ATIVO, PASSIVO
CampaignType	EMAIL, SOCIAL, PPC, SEO, EVENT, WEBINAR, DIRECT_MAIL, OTHER
MessageChannel	EMAIL, WHATSAPP, SMS, CALL, INSTAGRAM
MessageDirection	INBOUND, OUTBOUND
ContactType	LAWYER, JUDGE, CLIENT, TECHNICAL_ASSISTANT, OTHER
Role	SUPER_ADMIN, ADMIN, MANAGER, SALES, SUPPORT, USER, VIEWER

`Project.status`, `Invoice.status` e `Contract.status` são strings livres, não enums — não há validação de valores permitidos no backend.

Views analíticas

Criadas em `V25__create_analytics_views.sql` e consultadas por `AnalyticsViewRepository`. Elas agregam dados para o dashboard sem penalizar as consultas transacionais.

View	Conteúdo
<code>vw_entity_counts</code>	Contagens por entidade para os cards do dashboard
<code>vw_deal_pipeline</code>	Distribuição de negócios por pipeline e estágio
<code>vw_lead_analytics</code>	Funil e desempenho de leads
<code>vw_proposal_analytics</code>	Métricas de propostas
<code>vw_monthly_financial</code>	Série mensal de receitas e despesas

View	Conteúdo
<code>vw_project_profitability</code>	Rentabilidade por projeto (orçamento × custo)

Migrations

Numeração `V<n>__descricao.sql`, aplicadas em ordem. `R__seed_data.sql` é uma migration **repetível**: reexecuta sempre que seu conteúdo muda, alimentando dados de demonstração.

Linha do tempo das versões:

Faixa	Escopo
V1	Organizações e usuários
V2–V8	Leads, clientes, pipelines, negócios, propostas, projetos, tarefas, agenda, financeiro, campanhas, notificações, suporte
V9–V17	Refinamento de leads e clientes, detalhes, LGPD, mensagens, prospects, parceiros, status
V18–V24	Token público de proposta, plano de contas, processos jurídicos e cronômetro, contratos, produtos, faturas, documentos
V25–V26	Views analíticas, vínculo de propostas com negócios e histórico de estágio
V27–V28	Tokens do Google; campos jurídicos e periciais da proposta

05 — API REST

Base: `/api/v1`. Documentação interativa em `/swagger-ui.html`; contrato OpenAPI em `/v3/api-docs`.

Convenções

Autenticação. Todo endpoint privado exige `Authorization: Bearer <jwt>`. O tenant é lido do claim `organizationId` do próprio token — não é preciso enviá-lo no corpo nem na URL.

CRUD padrão. A maioria dos módulos expõe exatamente cinco operações:

Método	Path	Resposta
GET	<code>/api/v1/<recurso></code>	<code>Page<XxxResponse></code>
GET	<code>/api/v1/<recurso>/{id}</code>	<code>XxxResponse</code>
POST	<code>/api/v1/<recurso></code>	<code>XxxResponse</code> (200)
PUT	<code>/api/v1/<recurso>/{id}</code>	<code>XxxResponse</code>
DELETE	<code>/api/v1/<recurso>/{id}</code>	<code>204 No Content</code> (soft delete)

Paginação. As listagens aceitam os parâmetros padrão do Spring Data:

```
GET /api/v1/clients?page=0&size=10&sort=name,asc
```

A resposta é o envelope `Page` do Spring: `content`, `totalElements`, `totalPages`, `number`, `size`, `first`, `last`.

Erros. Formato uniforme, produzido pelo `GlobalExceptionHandler`:

```
{
  "error": "Erro de validação de dados",
  "details": ["email: não deve estar em branco"],
  "status": 400,
  "timestamp": "2026-08-07T10:00:00"
}
```

Autenticação e usuários

Método	Endpoint	Descrição
POST	<code>/auth/register</code>	Cria organização + usuário <code>ADMIN</code> e devolve o token
POST	<code>/auth/login</code>	Autentica e devolve o token

Método	Endpoint	Descrição
GET	/users /users/{id}	Lista e consulta usuários da organização
GET	/users/me	Dados do usuário autenticado
POST PUT DELETE	/users /users/{id}	Gestão de usuários

CRM

Recurso	Base	Endpoints adicionais
Prospects	/prospects	POST /{id}/promote — promove prospect a lead
Parceiros	/partners	—
Leads	/leads	POST /{id}/convert — converte em cliente; POST /{id}/recalculate-score
Notas de lead	/leads/{leadId}/notes	DELETE /{noteId}
Timeline de lead	/leads/{leadId}/timeline	Somente leitura
Cientes	/clients	—
Notas de cliente	/clients/{clientId}/notes	DELETE /{noteId}
Anexos de cliente	/clients/{clientId}/attachments	GET /{attachmentId}/download, DELETE /{attachmentId}
Timeline de cliente	/clients/{clientId}/timeline	Somente leitura
Contatos	/contacts	—
Pipelines	/pipelines	GET /{pipelineId}/stages, POST /stages, PUT /stages/{stageId}, DELETE /stages/{stageId}
Negócios	/deals	POST /{id}/transition, POST /{id}/mark-lost, POST /{id}/reopen, POST /{id}/convert-to-project
Comunicações	/communications	GET /lead/{leadId}, GET /client/{clientId}
Campanhas	/campaigns	POST /{id}/send

Operações

Recurso	Base	Endpoints adicionais
Propostas	<code>/proposals</code>	<code>GET /public/{token}</code> (público), <code>GET /{id}/pdf</code> , <code>GET /public/{token}/pdf</code> (público), <code>POST /{id}/convert-to-project</code>

O corpo de `ProposalRequest` aceita, além dos campos comerciais, o bloco jurídico e pericial: `lawyerContactId`, `lawyerName`, `referralSource`, `expertUserId`, `technicalManagerUserId` e `projectId`. A resposta devolve `lawyerContactId`, `lawyerName` (resolvido a partir do contato quando houver), `referralSource`, `expertUser`, `technicalManagerUser`, `projectId` e `projectName`. Ver [Regras de negócio](#). | Contratos | `/contracts` | — || Projetos | `/projects` | — || Produtos | `/products` | — || Tarefas | `/tasks` | — || Agenda | `/calendar-events` | — || Processos jurídicos | `/legal-processes` | `GET /search` || Documentos | `/documents` | — || Tickets | `/support-tickets` | — || Apontamento de horas | `/time-entries` | — |

Financeiro

Recurso	Base	Endpoints adicionais
Faturas	<code>/invoices</code>	<code>GET /{id}/pdf</code> , <code>GET /report/pdf</code>
Transações	<code>/financial-transactions</code>	—
Plano de contas	<code>/chart-of-accounts</code>	<code>GET /tree</code> (hierarquia), <code>POST /import</code>
Contas bancárias	<code>/bank-accounts</code>	—
Comissões	<code>/commissions</code>	—
Regras de comissão	<code>/commission-rules</code>	—
Relatórios financeiros	<code>/financial-reports</code>	<code>GET /cash-flow</code> , <code>GET /income-statement</code>
Relatórios	<code>/reports</code>	<code>GET /dre</code> , <code>GET /dfc</code>

Dashboard e analytics

Método	Endpoint	Conteúdo
<code>GET</code>	<code>/dashboard/summary</code>	Resumo consolidado
<code>GET</code>	<code>/dashboard/counts</code>	Contadores por entidade
<code>GET</code>	<code>/dashboard/sales</code>	Métricas de vendas
<code>GET</code>	<code>/dashboard/leads</code>	Funil de leads

Método	Endpoint	Conteúdo
GET	/dashboard/financial-trend	Série mensal financeira
GET	/dashboard/projects	Rentabilidade de projetos
GET	/dashboard/proposals	Métricas de propostas
GET	/analytics/dashboard	Agregação direta das views analíticas

Transversais

Recurso	Base	Observação
Notificações	/notifications	PUT /{id}/read marca como lida
Logs de auditoria	/audit-logs	Somente leitura
LGPD	/lgpd	POST /consent, GET /export, DELETE /forget
Integrações	/integrations	CRUD + /google/status, /google/connect, /google/disconnect, /google/calendar, /ai/generate

Os endpoints /integrations/google/* e /integrations/ai/generate no controller devolvem respostas **mockadas** e não usam o GoogleIntegrationService real. Veja **Pontos de atenção**.

Endpoints públicos

Dois fluxos funcionam sem autenticação:

Proposta pública. GET /api/v1/proposals/public/{token} e GET /api/v1/proposals/public/{token}/pdf localizam a proposta pelo publicToken (UUID). O link é entregue ao cliente final e renderizado pela rota /public/proposals/:token da SPA. Quem tiver o token acessa a proposta — não há expiração de token nem segundo fator.

Saúde da aplicação. GET /actuator/health e GET /actuator/info, com show-details: never.

06 — Frontend

SPA Angular 18 com componentes standalone, Angular Material 18 e Chart.js. Todo o conteúdo é lazy-loaded por módulo.

Estrutura de rotas

/login	público
/register	público
/public/proposals/:token	público – proposta enviada ao cliente
/portal/**	portais de cliente e parceiro
/	ShellComponent – protegido por authGuard (rota padrão)
— dashboard	
— profile	
— 28 módulos de negócio	
— users	+ adminGuard
— integrations	+ adminGuard
**	redireciona para /

Shell

ShellComponent é o layout do workspace autenticado: sidebar recolhível, toolbar com breadcrumbs, notificações com polling, alternância de tema claro/escuro, menu do usuário e cronômetro global (**TimerComponent**) para apontamento de horas.

A navegação é agrupada em quatro seções:

Seção	Itens
(topo)	Dashboard
CRM	Prospecção, Agenda, Parceiros / Indicadores, Clientes, Contatos, Leads, Negócios, Pipelines
Operações	Projetos, Produtos, Contratos, Processos, Tarefas, Propostas, Campanhas, Tickets, Documentos
Financeiro	Faturamento, Transações, Plano de Contas, Relatórios, Contas Bancárias, Horas, Comissões
Admin	Usuários, Integrações

O shell também traz um tour de onboarding com passos guiados (boas-vindas, menu de módulos, notificações, tema, menu do usuário).

O rótulo do produto no shell é "IBCAPPA CRM"; o nome oficial é **Axel CRM** (ver **PRODUCT.md**).

Componentes compartilhados

ListPageComponent

Base de praticamente todas as telas de listagem. Recebe dados via `@Input` e emite intenções via `@Output` — não conhece nenhum módulo específico.

Entrada	Função
<code>title</code>	Título da página
<code>columns</code>	<code>ColumnDef[]</code> — <code>{ key, label }</code>
<code>data</code>	Linhas da página atual
<code>totalElements</code> , <code>pageSize</code>	Paginação server-side
<code>loading</code> , <code>error</code>	Estados de carregamento e falha
<code>emptyMessage</code> , <code>emptyIcon</code> , <code>emptyActionLabel</code>	Estado vazio
<code>kpis</code>	<code>KpiDef[]</code> — <code>{ label, value, icon, color }</code>

Saída	Disparo
<code>pageChange</code> , <code>sortChange</code>	Paginação e ordenação
<code>add</code> , <code>edit</code> , <code>remove</code> , <code>view</code>	Ações de linha e da página
<code>retry</code>	Nova tentativa após erro

A coluna `actions` é anexada automaticamente ao fim das colunas declaradas.

FormDialogComponent

Diálogo genérico de formulário usado para criar e editar registros, dirigido por configuração de campos.

TimelineComponent

Renderiza a linha do tempo de leads e clientes, alimentada por `TimelineService`.

Camada core

Arquivo	Função
<code>core/guards/auth.guard.ts</code>	Exige token; redireciona para <code>/login</code>
<code>core/guards/admin.guard.ts</code>	Exige papel <code>ADMIN</code> ou <code>SUPER_ADMIN</code> ; redireciona para <code>/dashboard</code>

Arquivo	Função
<code>core/interceptors/auth.interceptor.ts</code>	Injeta o Bearer token; em 401 faz logout
<code>core/interceptors/tenant.interceptor.ts</code>	Injeta o header <code>X-Tenant-Id</code>
<code>core/services/auth.service.ts</code>	Login, registro, logout, usuário corrente (<code>BehaviorSubject</code>)
<code>core/services/base.service.ts</code>	CRUD HTTP genérico sobre <code>environment.apiUrl</code>
<code>core/services/client-detail.service.ts</code>	Agregação da tela de detalhe do cliente
<code>core/services/lead-detail.service.ts</code>	Agregação da tela de detalhe do lead
<code>core/services/timeline.service.ts</code>	Timeline de leads e clientes
<code>core/models/models.ts</code>	Interfaces compartilhadas (<code>Page<T></code> , <code>User</code> , DTOs)

`BaseService<T>`

```

getPage(path, page = 0, size = 10, sort = 'id,asc'): Observable<Page<T>>
getById(path, id): Observable<T>
create(path, body): Observable<T>
update(path, id, body): Observable<T>
delete(path, id): Observable<void>

```

Serviços de módulo estendem essa classe e fixam o path do recurso, o que mantém as chamadas HTTP em um único ponto.

Telas com layout próprio

A maioria dos módulos usa o par lista + diálogo. Estas fogem do padrão:

Tela	Particularidade
<code>dashboard</code>	KPIs, gráficos Chart.js, funil de leads, tarefas e comissões recentes
<code>clients/client-detail</code>	Abas com notas, anexos, contatos e timeline
<code>leads/lead-detail</code>	Detalhe com score, timeline e comunicações
<code>projects/project-detail</code>	Detalhe com dados de entrega e financeiro
<code>proposals/public-proposal</code>	Visualização pública por token, fora do shell
<code>portal/client-portal</code> , <code>portal/partner-portal</code>	Portais externos
<code>reports</code>	Relatórios DRE e DFC

Tela	Particularidade
<code>integrations</code>	Configuração de integrações (área admin)
<code>profile</code>	Perfil do usuário

Configuração de ambiente

`src/environments/environment.ts` (produção) e `environment.development.ts` (local) expõem `apiUrl`. No deploy do Render, o build substitui o placeholder `__API_URL__` pelo valor da variável `API_URL` antes de compilar.

```
export const environment = {  
  production: false,  
  apiUrl: 'http://localhost:8080/api/v1'  
};
```

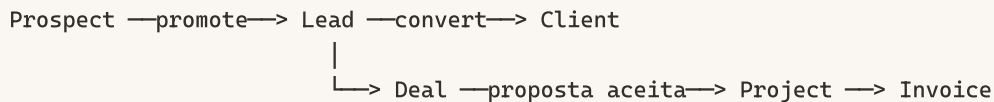
Design system

Tokens de cor, tipografia, raios e espaçamentos estão em `DESIGN.md`. Em resumo: tema escuro por padrão (`:root`), tema claro em `[data-theme="light"]`, laranja `#FC6E20` como única cor de ação da marca, tipografia Outfit (estrutura) + Inter (dados), Material Icons Round.

07 — Regras de negócio

Este documento descreve os fluxos que envolvem mais do que CRUD. Para os demais módulos, o comportamento é o CRUD padrão descrito em [API REST](#).

Funil comercial



Prospect → Lead

`POST /api/v1/prospects/{id}/promote` (`ProspectService.promoteToLead`)

Copia nome, e-mail, telefone, empresa, origem e anotações para um novo `Lead` no estágio `NEW`. O prospect passa a apontar para o lead criado (`convertedLead`) e recebe `convertedAt`. Um prospect já convertido não pode ser promovido de novo.

Lead → Client

`POST /api/v1/leads/{id}/convert` (`LeadConversionService.convertLeadToClient`)

Cria um `Client` a partir do lead (nome, e-mail, telefone, empresa → `companyName`, anotações, responsável), marca o lead como `CONVERTED`, grava `convertedAt` e o vínculo `convertedClient`, e registra um log de auditoria com ação `CONVERT`.

A operação é **idempotente**: se o lead já está `CONVERTED` e tem cliente vinculado, o cliente existente é devolvido em vez de um duplicado.

Lead scoring

`POST /api/v1/leads/{id}/recalculate-score` (`LeadScoringService.recalculate`)

O score é recalculado somando quatro componentes e depois limitado ao intervalo **0–100**.

1. Valor estimado — 1 ponto a cada R\$ 100 (arredondamento para baixo).

2. Qualidade da origem:

Origem	Pontos
<code>REFERRAL</code>	30
<code>EVENT</code>	25
<code>WEBSITE</code>	20

Origem	Pontos
SOCIAL_MEDIA	15
EMAIL	10
PHONE	5
demais	8

3. Contato recente (`lastContactAt`): até 3 dias → +20; até 7 dias → +10; até 30 dias → +5.

4. Recência do cadastro (`createdAt`): até 7 dias → +15; até 30 dias → +10; até 90 dias → +5.

O cálculo é sob demanda — não há job periódico recalculando scores.

Motor de pipeline

`PipelineEngine` centraliza as mudanças de estágio de um negócio e é a única via que grava `deal_stage_history`. Todas as operações são transacionais e geram log de auditoria.

Transição de estágio

`POST /api/v1/deals/{id}/transition` — corpo com `targetStageId` e `reason` opcional.

Regras aplicadas, nesta ordem:

1. Negócio já **ganho** não pode mudar de estágio.
2. Negócio já **perdido** precisa ser reaberto antes.
3. O estágio de destino precisa pertencer ao **mesmo pipeline** do negócio.
4. Movimento **regressivo** (posição de destino menor que a atual) exige justificativa em `reason` — sem ela, `400`.

Executadas as validações, o motor:

1. fecha o registro aberto em `deal_stage_history`, preenchendo `leftAt` e `durationSeconds`;
2. move o negócio para o estágio de destino;
3. se o destino for o **último estágio do pipeline**, marca `won = true` e preenche `closedAt`;
4. abre um novo registro de histórico com `enteredAt`, motivo e autor;
5. registra auditoria `STAGE_TRANSITION` com o estágio anterior e o novo.

Marcar como perdido

`POST /api/v1/deals/{id}/mark-lost` — recusa negócios já ganhos. Fecha o histórico corrente, define `won = false` e `closedAt`, e audita `DEAL_LOST` com o motivo.

Reabrir

`POST /api/v1/deals/{id}/reopen` — recusa negócios que não estão fechados. Zera `won` e `closedAt`, abre novo registro de histórico no estágio atual e audita `DEAL_REOPENED`.

Duração por estágio

Cada linha de `deal_stage_history` guarda `enteredAt`, `leftAt` e `durationSeconds`, o que permite medir tempo médio por estágio e identificar gargalos do funil.

Propostas

Cálculo do total

Ao criar ou atualizar, o total é a soma dos itens menos `discountAmount`. Os itens são persistidos junto (`proposal_items`).

Campos jurídicos e periciais

A proposta registra o contexto do caso além do comercial:

Campo	Regra
Advogado vinculado	FK para um <code>Contact</code> da organização. Omitir o campo no request limpa o vínculo; contato inexistente resulta em <code>404</code> .
Nome do advogado	Texto livre, gravado apenas quando não há contato vinculado. Com contato, o campo é zerado e a leitura devolve o nome do contato.
Origem da indicação	Valor de <code>LeadSource</code> . Quem indicou continua sendo o <code>partner</code> , que carrega a taxa de comissão.
Perito responsável	FK para <code>User</code> . Registro de responsabilidade — não entra no rateio de comissão.
Responsável técnico	FK para <code>User</code> . Mesma regra do perito.
Projeto vinculado	FK para um <code>Project</code> da organização.

Todos seguem a convenção dos demais relacionamentos da proposta: campo ausente no request limpa o valor atual.

Aprovação e ações automáticas

Quando o status passa a `ACCEPTED`, o serviço grava `approvedAt` e dispara `triggerPostApprovalActions`:

1. **Avança o negócio vinculado** — se a proposta tem `deal`, o negócio é movido para o **último estágio do pipeline** com o motivo `"Aprovação automática via Proposta comercial"`. Como é o estágio final, o negócio é marcado como ganho.

2. **Cria o projeto** — se a proposta **já tem um projeto vinculado**, nada é criado. Caso contrário, e se ainda não existir projeto com `sourceProposalId` igual ao da proposta, um é criado com:

- nome `"Projeto: <título da proposta>"`
- descrição, cliente e responsável herdados da proposta
- `startDate` = hoje, `endDate` = hoje + 3 meses
- `budget` = total da proposta, `cost` = 0
- `status` = `"PLANEJAMENTO"`

O projeto criado passa a ser o **projeto vinculado** da proposta.

Duas proteções contra duplicação: o vínculo direto (`Proposal.project`) e a checagem por `sourceProposalId`, que cobre propostas salvas mais de uma vez como aceitas.

Conversão manual

`POST /api/v1/proposals/{id}/convert-to-project` faz a mesma criação de projeto e vincula o resultado à proposta. Responde `400` quando:

- a proposta não está `ACCEPTED` — *"A proposta precisa estar aprovada (ACCEPTED) para ser convertida em projeto."*
- a proposta já tem projeto vinculado — *"Esta proposta já possui um projeto vinculado."*

`POST /api/v1/deals/{id}/convert-to-project` cria o projeto a partir de um negócio (`ProjectService.createFromDeal`).

Link público

Cada proposta recebe um `publicToken` (UUID). O cliente acessa `GET /api/v1/proposals/public/{token}` (ou `/pdf`) sem autenticação, e a SPA renderiza em `/public/proposals/:token`. O token não expira e não há segundo fator.

PDF

`ProposalService` gera o PDF com OpenPDF, tanto na rota autenticada quanto na pública. `InvoiceService` faz o mesmo para faturas (`/invoices/{id}/pdf` e `/invoices/report/pdf`).

Comissões

Existem dois caminhos de apuração.

Manual / por regra. Ao criar ou atualizar uma comissão, se `amount` não vier informado e houver uma `CommissionRule` aplicável, o valor é `dealValue × rule.percentage`.

Multinível a partir da proposta. `calculateCommissionsForTransaction` roda a partir de uma transação financeira e distribui a comissão entre até quatro papéis definidos na proposta:

Papel	Beneficiário	Percentual
CAPTURE	captureUser	captureRate
SELLER	sellerUser	sellerRate
PARTNER	partner	partnerRate
COLLABORATOR	collaboratorUser	collaboratorRate

Cada comissão vale $\text{valor da transação} \times \text{taxa do papel}$ e só é criada quando o beneficiário existe e a taxa é maior que zero. Como a base é o valor da transação, comissões são apuradas conforme o dinheiro entra, não no fechamento do negócio.

`payCommission` marca a comissão como paga.

Apontamento de horas

`TimeEntryService` aceita `durationMinutes` explícito. Quando ele não vem, mas `startTime` e `endTime` estão preenchidos, a duração é calculada automaticamente pela diferença entre os dois. O shell tem um cronômetro global (`TimerComponent`) para alimentar esse registro durante o trabalho.

Campanhas

`POST /api/v1/campaigns/{id}/send` monta a lista de destinatários (`campaign_recipients`) a partir de **todos** os leads e clientes da organização que tenham e-mail ou telefone, e marca a campanha como enviada. Uma campanha já enviada não pode ser reenviada.

O envio é uma **simulação**: os destinatários são gravados, mas nenhuma mensagem é efetivamente disparada — não há integração de e-mail ou WhatsApp conectada a esse fluxo. Veja [Pontos de atenção](#).

Plano de contas

`GET /api/v1/chart-of-accounts/tree` devolve a hierarquia montada de contas; `POST /import` importa um plano padrão. Os tipos disponíveis são `RECEITA`, `DESPESA`, `ATIVO` e `PASSIVO`.

Relatórios financeiros

Endpoint	Relatório
<code>/financial-reports/cash-flow</code>	Fluxo de caixa
<code>/financial-reports/income-statement</code>	Demonstração de resultado
<code>/reports/dre</code>	DRE

Endpoint	Relatório
<code>/reports/dfc</code>	DFC

Os números do dashboard vêm das views analíticas descritas em [Modelo de dados](#).

08 — Ambiente e deploy

Pré-requisitos

Ferramenta	Versão
Java (JDK)	21
Node.js	20+
PostgreSQL	16+
Maven	3.9+ instalado — o repositório não traz o wrapper <code>mvnw</code>

Execução local

Banco

```
createdb axelcrm
```

O `application.yml` aponta para `jdbc:postgresql://localhost:5432/axelcrm` com usuário `postgres`. Ajuste a senha ou sobrescreva via variável de ambiente conforme seu ambiente local.

Backend

```
cd backend
mvn spring-boot:run
```

API em `http://localhost:8080`. O Flyway aplica as migrations na inicialização; não é necessário criar tabelas manualmente.

Para logs SQL e de segurança mais verbosos, use o perfil `dev`:

```
mvn spring-boot:run -Dspring-boot.run.profiles=dev
```

Frontend

```
cd frontend
npm install
npm start      # equivale a ng serve
```

App em `http://localhost:4200`, apontando para a API conforme `src/environments/environment.ts`:

```
export const environment = {  
  production: false,  
  apiUrl: 'http://localhost:8080/api/v1'  
};
```

Primeiro acesso

Acesse `/register` e crie a primeira conta. O registro cria a organização e o usuário `ADMIN` dela.

Documentação da API em execução

Recurso	URL local
Swagger UI	<code>http://localhost:8080/swagger-ui.html</code>
OpenAPI JSON	<code>http://localhost:8080/v3/api-docs</code>
Health check	<code>http://localhost:8080/actuator/health</code>

Perfis de configuração

Arquivo	Uso
<code>application.yml</code>	Padrão local: PostgreSQL em localhost, JWT com segredo de desenvolvimento, logs de segurança e web em <code>DEBUG</code>
<code>application-dev.yml</code>	Acrescenta <code>show-sql</code> , SQL formatado e log <code>DEBUG</code> de <code>com.axelcrm</code>
<code>application-render.yml</code>	Produção no Render: conexão e segredo via variáveis de ambiente, logs em <code>INFO</code>

Configurações fixas em todos os perfis: `spring.jpa.hibernate.ddl-auto: none` e Flyway habilitado com `baseline-on-migrate: true`.

Variáveis de ambiente

Backend

Variável	Perfil	Descrição
<code>SPRING_PROFILES_ACTIVE</code>	todos	Perfil ativo (<code>render</code> em produção)
<code>PORT</code>	render	Porta HTTP (padrão 8080)
<code>DB_HOST</code>	render	Host do PostgreSQL

Variável	Perfil	Descrição
<code>DB_PORT</code>	render	Porta do PostgreSQL
<code>DB_NAME</code>	render	Nome do banco
<code>DB_USER</code>	render	Usuário
<code>DB_PASSWORD</code>	render	Senha
<code>JWT_SECRET</code>	render	Segredo de assinatura HMAC256 — obrigatório , definido manualmente
<code>JWT_EXPIRATION</code>	render	Validade do token em ms (padrão <code>86400000</code>)
<code>google.client-id</code> / <code>google.client-secret</code> / <code>google.redirect-uri</code>	opcional	OAuth2 do Google (<code>GoogleIntegrationService</code>)

O `application.yml` traz um `jwt.secret` de desenvolvimento embutido no repositório. Ele nunca deve ser usado em produção — em produção, `JWT_SECRET` é obrigatório e não tem valor padrão.

Frontend

Variável	Descrição
<code>API_URL</code>	URL base da API; substitui o placeholder <code>__API_URL__</code> durante o build

Migrations

Local: `backend/src/main/resources/db/migration/`.

Nomenclatura. `V<n>__descricao_em_snake_case.sql` para migrations versionadas; `R__nome.sql` para repetíveis.

Execução. Além do Flyway automático do Spring Boot, `FlywayMigrationConfig` implementa `CommandLineRunner` e executa `flyway.repair()` seguido de `flyway.migrate()` na subida da aplicação. O `repair()` corrige checksums divergentes no histórico — o que também significa que alterações em migrations já aplicadas passam despercebidas em vez de falhar. Nunca edite uma migration já aplicada; crie uma nova.

Ao criar uma migration:

1. Use o próximo número de versão livre — duas migrations com a mesma versão impedem a aplicação de subir. A última versão em uso é a `V28`.
2. Inclua `organization_id UUID NOT NULL` com FK para `organizations` em toda tabela de negócio.
3. Inclua `created_at`, `updated_at` e `deleted_at`.
4. Crie índice em `organization_id` — todas as consultas filtram por ele.

Deploy no Render

O `render.yaml` na raiz é um Blueprint que provisiona três recursos:

Recurso	Nome	Tipo
Banco	<code>axel-crm-db</code>	PostgreSQL (plano free)
API	<code>axel-crm-api</code>	Web service Docker, <code>rootDir: backend</code>
SPA	<code>axel-crm-app</code>	Static site, <code>rootDir: frontend</code>

As credenciais do banco são injetadas automaticamente no serviço da API via `fromDatabase`. `JWT_SECRET` **está marcado como `sync: false`** e precisa ser preenchido manualmente no painel do Render — sem ele a aplicação não sobe.

O build do frontend é:

```
npm ci && sed -i "s|__API_URL__|$API_URL|g" src/environments/environment.ts \  
&& npm run build -- --configuration production
```

O diretório publicado é `dist/crm-axel-frontend/browser`, com rewrite de `/*` para `/index.html` (necessário para o roteamento client-side do Angular).

URLs esperadas

Serviço	URL
API	<code>https://axel-crm-api.onrender.com</code>
Swagger UI	<code>https://axel-crm-api.onrender.com/swagger-ui.html</code>
Frontend	<code>https://axel-crm-app.onrender.com</code>

Imagem Docker do backend

`backend/Dockerfile` usa build multi-stage: `maven:3.9-eclipse-temurin-21` compila o JAR com testes desativados (`-DskipTests`), e `eclipse-temurin:21-jre` executa apenas o artefato final, expondo a porta 8080.

09 — Guia de desenvolvimento

Como adicionar um módulo CRUD completo

O sistema tem mais de 30 módulos que seguem o mesmo caminho. Use `Client` como referência viva — ele é o exemplo mais limpo do padrão.

Backend

1. Migration. Crie `V<n>__create_<tabela>.sql` com `id UUID PK`, `organization_id UUID NOT NULL` (FK para `organizations`), as colunas do domínio, `created_at`, `updated_at`, `deleted_at` e índice em `organization_id`.

2. Entidade. Em `entity/`, estenda `BaseEntity`:

```
@Entity
@Table(name = "widgets")
@Data
@EqualsAndHashCode(callSuper = true)
public class Widget extends BaseEntity {
    @Column(nullable = false)
    private String name;
}
```

3. Repositório. Em `repository/`, com o filtro de tenant no nome dos métodos:

```
public interface WidgetRepository extends JpaRepository<Widget, UUID> {
    Page<Widget> findByOrganization_IdAndDeletedAtIsNull(UUID organizationId, Pageable
        pageable);
    Optional<Widget> findByIdAndOrganization_IdAndDeletedAtIsNull(UUID id, UUID
        organizationId);
}
```

4. DTOs. Em `dto/`, dois records: `WidgetRequest` (com Jakarta Validation) e `WidgetResponse`.

5. Service. Em `service/`, com `@Transactional`, recebendo `organizationId` como primeiro parâmetro em toda operação. `delete` preenche `deletedAt` — nunca remove a linha.

6. Controller. Em `controller/`, fino, extraindo o tenant do `TenantContext` e anotado para o Swagger:

```
@RestController
@RequestMapping("/api/v1/widgets")
@RequiredArgsConstructor
@Tag(name = "Widgets", description = "Endpoints for managing widgets")
public class WidgetController { ... }
```

Frontend

- 7. Service.** Estenda `BaseService<Widget>` e fixe o path `widgets`.
- 8. Componente de lista.** Componente standalone que usa `ListPageComponent`, declarando `columns`, `kpis` e ligando os outputs a chamadas do service e ao `FormDialogComponent`.
- 9. Rotas do módulo.** `features/widgets/widgets.routes.ts` exportando `WIDGETS_ROUTES`.
- 10. Registro.** Adicione a rota lazy em `app.routes.ts` e o item de navegação na seção correta de `shell.component.ts`.

Convenções obrigatórias

Toda consulta filtra por organização. Um repositório sem o filtro de `organization_id` vaza dados entre tenants. Não há rede de proteção no banco.

Toda exclusão é lógica. Preencha `deletedAt` e filtre por `DeletedAtIsNull` em todas as leituras.

Entidade nunca sai pela API. Controllers respondem DTOs; a serialização de entidades JPA com relacionamentos lazy quebra ou expõe dados demais.

O tenant vem do token. Nunca aceite `organizationId` vindo do corpo da requisição ou de query string.

Regra de negócio fica no service. Controllers só orquestram; repositórios só consultam.

Mensagens de erro de negócio em português. Elas chegam ao operador. Use `BadRequestException` para violação de regra e `ResourceNotFoundException` para registro inexistente — o `GlobalExceptionHandler` cuida do status HTTP.

Testes

Backend

```
cd backend
mvn test
```

Stack: JUnit 5 (`spring-boot-starter-test`), Mockito, `spring-security-test` e H2 em memória.

São **144 testes** em 23 classes: testes de controller (autenticação, usuários, clientes, negócios, projetos, propostas, dashboard, financeiro) e testes unitários de service (`ProposalService`, `PipelineEngine`, `AnalyticsService`, `DealService`, `LeadService`, `ProjectService`, `ClientService`, `LegalProcessService`, entre outros), apoiados em `config/BaseServiceTest.java`.

Ainda sem cobertura: `LeadScoringService`, `CommissionService`, `LgpdService`, `CampaignService` e a maior parte dos módulos de CRUD simples.

O build Docker roda com `-DskipTests`; os testes precisam ser executados no CI ou localmente.

Frontend

```
cd frontend
npm test           # Karma + Jasmine
```

Não há specs relevantes no repositório no momento.

Build

```
# backend - JAR executável em target/
cd backend && mvn package

# frontend - bundle em dist/crm-axel-frontend/browser
cd frontend && npm run build -- --configuration production
```

Commits

Formato convencional: `<tipo>: <descrição>`, com tipos `feat`, `fix`, `refactor`, `docs`, `test`, `chore`, `perf`, `ci`. Descrição no imperativo, explicando o efeito da mudança.

Onde procurar cada coisa

Preciso de...	Vá para
Contrato de um endpoint	Swagger UI, ou <code>controller/</code> + <code>dto/</code>
Regra de negócio	<code>service/</code>
Estrutura de tabela	<code>resources/db/migration/</code>
Isolamento de tenant	<code>auth/security/TenantFilter</code> , <code>TenantContext</code> , <code>commons/entity/BaseEntity</code>
Autenticação	<code>auth/security/</code> , <code>auth/service/AuthService</code>
Tela de listagem	<code>shared/list-page/</code> , mais o componente do módulo
Navegação	<code>app.routes.ts</code> , <code>shell/shell.component.ts</code>
Tokens visuais	<code>DESIGN.md</code>

10 — Pontos de atenção

Itens verificados diretamente no código em 07/08/2026. Não são especulações: cada um aponta o arquivo onde o comportamento está. A ordem é por severidade.

Resolvido desde a primeira versão deste documento: a duplicidade de versão entre

`V26__create_google_tokens.sql` e `V26__proposals_link_and_deal_stage_history.sql`, que impedia o Flyway de subir. O primeiro passou a ser `V27`. Ao provisionar um ambiente que já tinha a versão antiga aplicada, confira `flyway_schema_history` antes.

Crítico

A API não aplica autorização por papel

Nenhum controller usa `@PreAuthorize`, `@Secured` ou regras por papel no `SecurityConfig` — a única condição é `anyRequest().authenticated()`. O papel entra no token como authority, mas nada o consome no backend.

O `adminGuard` do frontend esconde `/users` e `/integrations` de não-administradores, mas isso é apenas navegação: qualquer usuário autenticado pode chamar `POST /api/v1/users` ou os endpoints de integração diretamente e criar contas, alterar papéis ou alterar configurações.

Correção: aplicar `@PreAuthorize("hasAnyAuthority('ADMIN','SUPER_ADMIN')")` nos controllers administrativos e habilitar `@EnableMethodSecurity`.

Segredo JWT de desenvolvimento versionado

`application.yml` traz `jwt.secret` embutido no repositório. Ele é o padrão do perfil local; o perfil `render` exige `JWT_SECRET` por variável de ambiente. Ainda assim, um deploy que suba sem o perfil `render` ativo usaria um segredo público, permitindo forjar tokens de qualquer organização.

Correção: remover o valor padrão e falhar na inicialização quando `JWT_SECRET` não estiver definido.

Alto

`JWT_EXPIRATION` não tem efeito

`JwtUtil` injeta `@Value("${jwt.expiration-ms:86400000}")`, mas os arquivos de configuração definem `jwt.expiration`. A propriedade lida não existe, então o valor usado é sempre o default de 24 h — inclusive no Render, onde `JWT_EXPIRATION` está declarado no `render.yaml`.

Correção: alinhar os nomes (`jwt.expiration` no `@Value`, ou renomear a chave nos YAMLS).

CORS aberto com credenciais

`CorsConfig` combina `allowedOriginPatterns("*")` com `allowCredentials(true)` e `allowedHeaders("*")`. Qualquer origem pode chamar a API. Como a autenticação é por header `Authorization` (e não cookie), o risco imediato é menor, mas a configuração não deveria chegar a produção assim.

Correção: restringir a lista de origens à URL do frontend por perfil.

Integrações do Google são mocks no controller

`IntegrationController` responde `/google/status`, `/google/connect`, `/google/disconnect` e `/google/calendar` com dados fixos em memória — inclusive um e-mail hard-coded (`contato@axelpro.com.br`) e uma lista estática de eventos. `/ai/generate` devolve texto pré-fabricado.

Existe um `GoogleIntegrationService` funcional, com fluxo OAuth2 completo (authorization URL, troca de código, refresh token, leitura de Calendar e Contacts, envio via Gmail) e persistência em `google_tokens` — mas **o controller não o utiliza**.

Correção: ligar o controller ao service e configurar `google.client-id` / `google.client-secret` / `google.redirect-uri`. O endpoint de callback do OAuth2 (`/api/v1/integrations/google/callback`, referenciado como redirect URI) também não está implementado no controller.

Envio de campanha carrega todos os clientes do banco

`CampaignService.sendCampaign` usa `clientRepository.findAll()` e filtra a organização **em memória**, enquanto para leads usa a consulta filtrada correta. Não há vazamento de dados na resposta, mas a query traz clientes de todos os tenants para a JVM — o custo cresce com a base inteira, não com a organização.

Além disso, a campanha inclui **todos** os leads e clientes com e-mail ou telefone, sem qualquer segmentação, e o envio é simulado: os destinatários são gravados em `campaign_recipients` e a campanha é marcada como enviada, mas nenhuma mensagem sai.

Correção: trocar por `findByOrganization_IdAndDeletedAtIsNull` e deixar explícito na UI que o envio é simulado enquanto não houver provedor conectado.

Médio

Logs em DEBUG na configuração padrão

`application.yml` define `org.springframework.security` e `org.springframework.web` em `DEBUG`. É útil no desenvolvimento e ruidoso (e potencialmente revelador de detalhes de requisição) em qualquer outro lugar. O perfil `render` corrige para `INFO`.

`open-in-view` habilitado

`spring.jpa.open-in-view: true` mantém a sessão do Hibernate aberta durante a renderização da resposta. Isso mascara `LazyInitializationException` e favorece consultas N+1 disparadas na serialização.

`flyway.repair()` a cada inicialização

`FlywayMigrationConfig` executa `repair()` antes de `migrate()` toda vez que a aplicação sobe. Isso reescreve checksums divergentes em vez de falhar — ou seja, uma migration já aplicada que for editada passa despercebida, e os ambientes divergem silenciosamente.

Status como string livre

`Project.status`, `Invoice.status` e `Contract.status` são `String` sem validação, enquanto domínios equivalentes (`LeadStage`, `ProposalStatus`, `TaskStatus`) usam enums. Valores são gravados como vierem do cliente, sem garantia de consistência.

Cobertura de testes parcial

São 144 testes em 23 classes, cobrindo controllers dos fluxos principais e boa parte dos services de maior risco (`ProposalService`, `PipelineEngine`, `AnalyticsService`, `DealService`, `ProjectService`). Seguem sem cobertura `LeadScoringService`, `CommissionService`, `LgpdService` e `CampaignService` — todos com regra de negócio relevante. O build Docker roda com `-DskipTests`.

Exceções fora do padrão do handler

`ProspectService.promoteToLead` e `CampaignService.sendCampaign` lançam `IllegalStateException`, que não é tratada pelo `GlobalExceptionHandler` e vira `500 Internal Server Error` em vez de `400` com mensagem legível. As demais regras usam `BadRequestException` corretamente.

Baixo

Marca inconsistente na interface

O nome oficial do produto é **Axel CRM** (repositório, título da API, `PRODUCT.md`), mas o shell exibe "IBCAPPA CRM". `PRODUCT.md` classifica o rótulo como placeholder legado. Alinhar antes de qualquer material voltado a usuário.

Header `X-Tenant-Id` sem uso

O `tenantInterceptor` envia `X-Tenant-Id` em toda requisição, e o `CorsConfig` o expõe, mas o backend nunca o lê — o tenant vem exclusivamente do claim do JWT. O header é inofensivo, porém sugere um mecanismo de isolamento que não existe. Removê-lo evita que alguém confie nele no futuro.

Token de proposta pública sem expiração

`publicToken` é um UUID permanente. Quem obtiver o link acessa a proposta e o PDF indefinidamente, mesmo depois de expirada ou rejeitada. Considere data de validade ou revogação do token.

README desatualizado

O `README.md` marca "Calendar & tasks" e "File attachments" como pendentes, mas ambos existem (`/api/v1/calendar-events`, `/api/v1/tasks`, `/api/v1/clients/{id}/attachments`).